

问题回答

1 结题验收追问速答：模块划分、技术栈、算法与缺陷

日期：2026-05-20

用途：现场演示后的问答准备。回答时不要主动讲具体代码细节，优先讲设计思路、模块职责、控制流、算法边界和实验结论。

1.1 1. 项目整体怎么划分模块？

推荐回答：

我把系统按“输入、契约、规划、执行、工具、存储和前端展示”来划分。

后端 API 层负责任务创建、任务查询、HITL 决策、事件和报告接口；models 层负责统一数据契约，比如任务、计划、步骤结果、PendingAction、Decision、RuntimeState 和 DesignResult；agents 层负责多 Agent 协作，包括 Planner、Executor、Safety 和 Summarizer；workflow 层负责 FSM、执行推进、等待态、恢复、runtime state、候选重排和快照；adapters 层封装本地工具、远程 REST 服务和脚本差异；kg 层提供 ProteinToolKG，用来描述工具能力、I/O、成本、风险和安全约束；storage 层负责事件日志、快照和文件产物；前端 React 工作台负责展示任务状态、HITL 候选、报告、结构和事件时间线。

更短版：

系统核心是后端控制流。前端只做展示和提交决策；API 是边界；models 是契约；workflow 是状态机和恢复控制；agents 是职责分工；adapters 和 ToolKG 负责工具接入；storage 负责审计和恢复。

1.2 2. 为什么要这样划分？

推荐回答：

主要是为了把控制面和执行面分开。

Planner 只负责生成候选，不直接执行工具；Executor 负责执行已确认的步骤；Workflow/FSM 统一控制状态迁移；ToolAdapter 负责屏蔽具体工具差异；前端不直接推导或修改状态，只读取后端任务事实。

这样做可以保证系统在进入 WAITING_*、应用人工决策、失败恢复和最终完成时都有明确的状态边界和审计记录。

如果老师追问“这和普通前后端项目有什么不同”：

普通系统可能只是 CRUD 和页面展示；这个项目的重点是长链路科学工作流控制。任务会经历规划、执行、等待人工确认、局部修补、重规划、总结等状态，所以必须把 FSM、PendingAction、Decision、RuntimeState、EventLog 和 Snapshot 作为一等对象设计。

1.3 3. CEBRA-WP 算法在系统中落在哪里？

推荐回答：

CEBRA-WP 不是一个单独的新 Agent，也不是一个只在某个函数里的模块，而是贯穿候选生成、候选评分、运行时状态更新、运行时重排序和恢复动作选择的控制流算法。

Planner 生成候选集合和 score breakdown；WorkflowContext 维护 RuntimeState；RuntimeEvaluator 根据 runtime state 计算 runtime adjustment 和 action utility；PlanRunner / recovery 逻辑把动作建议映射为 continue、patch_local、suffix_replan 或 stop；PendingAction / Decision 把需要人工确认的动作接入 FSM。

一句话版：

CEBRA-WP 是工作流层的规划与恢复控制算法，工程上主要落在 workflow 运行时、候选评分、RuntimeEvaluator 和 HITL 决策链路中。

1.4 4. CEBRA-WP 和普通 LLM Planner 有什么区别？

推荐回答：

普通 LLM Planner 往往是给定目标后直接生成一条计划，然后执行。CEBRA-WP 不直接相信单条 LLM 输出，而是生成 Top-K 候选，先做工具存在性、schema、I/O、预算和安全过滤，再计算静态效用。

执行过程中，它还维护 Lite belief-state，根据 StepResult、SafetyResult、失败上下文和预算压力更新运行时状态，再对候选做运行时重排序，并在 continue、patch_local、suffix_replan、stop 之间选择恢复动作。

所以它解决的不是“让 LLM 写一条流程”，而是“在高代价、会失败、需要恢复的科学 workflows 中，如何更可控地选择和修正工具链”。

1.5 5. Lite belief-state 五个量分别是什么？

推荐回答：

Lite belief-state 是运行时的轻量状态估计，包含五个核心量。

p_success 表示当前链路继续执行后成功完成的估计倾向；p_structural_failure 表示结构性失败或后续必须重规划的风险；recovery_margin 表示保留已有前缀并继续修复的余量；expected_remaining_cost 表示从当前状态到结束的剩余成本暴露；evidence_sufficiency 表示已有证据是否足以支持进入更高代价步骤。

budget_pressure 是从 expected_remaining_cost 和预算上限派生出来的解释字段，不是五维主状态之一。

1.6 6. 页面里的数字是怎么计算的？

推荐回答：

现场 demo 的数字是固定 fixture，用于稳定展示界面和机制；真实系统中这些字段由同一套算法对象计算。

候选评分来自工具能力、I/O 契约、风险、成本、readiness 和恢复性；RuntimeState 来自 StepResult、SafetyResult、失败上下文、预算和进度；budget_pressure 由 expected_remaining_cost 和 budget_cap 派生；final_score 由 S_post 加 runtime adjustment 得到。

可以举 demo 里的例子：

这里 expected_remaining_cost 是 1.35，budget_cap 是 1.2，所以 budget_pressure 大约是 $1.35 / 1.2 = 1.12$ 。它说明当前预算压力已经偏高，所以系统更倾向于让人确认低成本 patch，而不是继续远程高代价调用。

1.7 7. 为什么需要 HITL？

推荐回答：

因为蛋白质设计 workflow 里有些动作代价高、风险高，并且错误后果可能影响后续多步。系统可以生成候选和推荐，但不应该自动替代人类批准关键决策。

所以我用 `WAITING_PLAN_CONFIRM`、`WAITING_PATCH_CONFIRM` 和 `WAITING_REPLAN_CONFIRM` 表示执行暂停，`PendingAction` 展示候选，`Decision` 表示人工决策。进入等待态前必须写事件和快照，人工决策后才允许继续推进。

一句话版：

HITL 不是前端按钮，而是 FSM 中的受控等待状态。

1.8 8. 为什么不用现成 workflow 引擎，比如 Nextflow？

推荐回答：

Nextflow 适合稳定、可复现的计算流水线，但这个项目还需要显式处理 `WAITING_*`、`PendingAction`、`Decision`、`RuntimeState`、`patch/replan` 和审计恢复。

这些是智能 workflow 控制面的语义，不只是任务调度语义。所以我保留了自定义 Workflow/FSM，把全局状态迁移和人工确认放在系统运行时里；具体工具执行仍可以通过 `ToolAdapter` 或外部脚本封装。

1.9 9. 用了哪些技术？

推荐回答：

后端主要使用 Python 3.12、FastAPI 和 Pydantic。Python 便于接入生物信息工具和模型服务；FastAPI 提供 API 边界；Pydantic 用来约束任务、计划、步骤结果、决策和报告等数据结构。

前端使用 React、TypeScript 和 Vite，主要负责 Dashboard、Task Builder、Task Detail、Pending Review、Report Explorer、Structure Viewer 和 Event Timeline。

系统还包括 ProteinToolKG、ToolAdapter、EventLog、TaskSnapshot，以及本地和远程工具服务适配。

1.10 10. 四个 Agent 怎么分工？

推荐回答：

`PlannerAgent` 负责生成 Plan、Patch 或 Replan 候选，不执行工具；`ExecutorAgent` 负责执行已确认的计划步骤和恢复流程；`SafetyAgent` 只输出 allow、warn 或 block 等风险判断，不直接改计划；`SummarizerAgent` 负责把已有结果和证据汇总为 `DesignResult` 和报告。

这个边界很重要，因为状态迁移由 Workflow/FSM 统一负责，Agent 不能越权直接把任务改成完成或失败。

1.11 11. 工具怎么接入？

推荐回答：

工具接入分两层。ProteinToolKG 描述工具能力、输入输出、成本、风险、安全约束和兼容关系；ToolAdapter 负责把统一的 PlanStep 输入转换成具体工具调用，并返回 outputs 和 metrics。

这样 Planner 看到的是工具能力和契约，Executor 看到的是统一适配器接口，不需要关心底层是本地 Python 脚本、命令行工具还是远程 REST 服务。

1.12 12. 报告浏览器的作用是什么？

推荐回答：

报告浏览器不是只展示自然语言总结，而是展示 DesignResult 的结构化内容，包括序列、结构路径、scores、objective scoring、top-k 候选后验结果、warnings、evidence refs 和 structure similarity。

它的作用是把结果、证据和审计路径连起来。老师如果问这个结论从哪里来，我可以指向 event_log、PDB 文件、候选评分和后验目标评分。

1.13 13. Event Timeline 的作用是什么？

推荐回答：

Event Timeline 展示任务从创建、规划、运行、进入等待态、人工决策、退出等待态，到总结和完成的全过程。

它证明系统不是黑箱给出结果，而是每次状态迁移、等待进入、决策应用和完成都可追溯。对于失败案例，也可以通过事件链分析失败来自工具异常、候选 I/O 不闭合、恢复循环还是安全阻断。

1.14 14. 实验结果应该怎么说？

推荐回答：

主实验是 12 个 task key、4 组策略、共 84 次运行。最终 81 次 DONE，3 次 FAILED。

static_top1 在当前矩阵中成功率是 100%，其他三组是 95.2%。所以我不会说 CEBRA-WP 显著提高最终成功率。

更准确的结论是：CEBRA-WP 的机制链路可执行、RuntimeState 可观测、HITL 和恢复决策可审计；fixed_threshold_gate 会带来额外高代价调用；lite_belief_state 提供了更完整的运行时解释信息。

1.15 15. 项目缺陷有哪些？

推荐回答：

第一，实验规模有限，84 runs 可以支撑机制分析，但统计效力不足。

第二，当前矩阵中真实 patch 主要出现在 fixed_threshold_gate 组，suffix_replan 和 terminal_stop 的矩阵级证据还不充分。

第三，dynamic_no_belief_state 和 lite_belief_state 在成功率和高代价调用均值上差异不明显，说明任务集对 belief-state 增量价值的放大能力不足。

第四，系统仍是原型，数据库持久化、ProteinToolKG 动态更新、远程服务故障切换、配额管理和外部 Agent 基线还可以继续加强。

1.16 16. 为什么 demo 不现场跑真实模型？

推荐回答：

现场演示的目标是稳定展示系统交互、FSM、HITL、结构查看和审计链路。真实模型和远程结构预测服务耗时长、受网络和服务状态影响，不适合作为现场实时依赖。

因此我使用本地确定性 fixture 展示机制，真实实验结果用第七章的冻结 84-run 矩阵和输出产物说明。

1.17 17. 如果老师问“创新点是什么”

推荐回答：

我认为创新点主要有四个。

第一，构建了一个可恢复、可审计的蛋白质设计 workflow 原型；第二，提出并实现 CEBRA-WP，把约束、证据、Lite belief-state 和恢复动作统一到 workflow 层；第三，把 FSM、HITL、EventLog 和 TaskSnapshot 结合起来，保证关键决策可追溯；第四，通过 84-run 消融矩阵分析机制价值和边界，没有把结论夸大成生物学功能验证。

1.18 18. 如果老师问“算法有什么不足”

推荐回答：

CEBRA-WP 当前是规则化、可解释的运行控制算法，不是学习到的最优策略。它的优点是可审计、可复现、容易落地；不足是权重和阈值仍带有工程启发式，需要更多任务和外部基线验证。

另外，当前 belief-state 能解释恢复决策，但在候选质量不足或 patch 循环时，仍可能无法自动打破循环。后续可以加入更强的 escalation 机制，让多次同类局部修补失败后更稳定地升级到 suffix_replan 或 terminal_stop。

1.19 19. 如果老师问“为什么是软工毕设而不是纯算法”

推荐回答：

这个课题的核心不是训练一个新的蛋白质模型，而是把多个模型、工具、远程服务和人工确认组织成一个可靠系统。

论文既有 CEBRA-WP 算法设计，也有明确的软件工程内容：需求分析、模块划分、API 和数据契约、FSM、HITL、前端工作台、工具适配、事件日志、快照恢复、测试和实验验证。

所以它更像是面向科研场景的智能 workflow 系统开发，而不是单一模型算法实验。

1.20 20. 最短总括回答

如果时间很紧，可以用这段总结：

这个系统的核心是一个受 FSM 约束的多 Agent 蛋白质设计 workflow。API 和 models 提供结构化任务和数据契约，Planner 生成候选，CEBRA-WP 根据静态分、后验目标证据和 Lite belief-state 做运行时重排序与恢复动作选择，Workflow/FSM 控制 WAITING_*、Decision、patch/replan 和 DONE，ToolAdapter 接入具体工具，EventLog 和 Snapshot 保证可追溯和可恢复。演示展示的是本地 fixture，真实结论来自 84-run 矩阵，主要证明机制可执行、可观测、可审计，而不是证明生物学功能。